

Selecting Model Complexity for Runge’s Function: Regularized Regression, Resampling and Gradient Descent

Sigurd Riseth
Department of Physics
University of Oslo
 Oslo, Norway
 sigurri@uio.no

Abstract—Runge’s function, the classic example of polynomial interpolation diverging with increasing degree, is a demanding test of how model complexity is chosen from noisy data. We fitted polynomials of degree up to 20 to 100 noisy samples with ordinary least squares, Ridge and Lasso regression, estimated the test error with the bootstrap and k -fold cross-validation, and replaced the closed-form solutions with our own gradient-descent code. Since the data were synthetic, we compared every estimate with the true error. The bootstrap and cross-validation selected median degrees of 8 and 10, against 14 for the true optimum, because their estimates are dominated by extrapolation beyond the resampled training range. Over 31 data sets, Ridge lowered the cross-validation error of least squares by a median of 5% but not the held-out error, and subgradient-descent Lasso did worse. Momentum and Adam reached the closed-form solution 25 and 9 times faster than plain gradient descent at degree 5, yet no optimizer converged at degree 10 or higher without regularization.

Index Terms—regression, regularization, resampling, bias-variance trade-off, gradient descent, automatic differentiation

I. INTRODUCTION

Polynomial regression is among the simplest nontrivial regression problems, yet it already exhibits the central tension in model selection: increasing the degree reduces the approximation error, but increases sensitivity to noise in the data. The Weierstrass approximation theorem [1] guarantees that any continuous function on a closed interval can be approximated uniformly to arbitrary accuracy by polynomials, but it says nothing about how to find such a polynomial from a finite set of noisy samples, or which degree to choose. Increasing the degree reduces the approximation error of the model class, yet it also makes the fitted coefficients increasingly sensitive to the particular data set. This tension is the bias–variance trade-off [2], [3]. Its lessons carry over essentially unchanged to the more flexible models used elsewhere in machine learning, which makes polynomial regression a transparent laboratory for them.

Runge’s function,

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1], \quad (1)$$

is the classical example of how this can fail. Although f is smooth, its polynomial interpolants on equispaced nodes

diverge near the endpoints as the degree increases; this is Runge’s phenomenon [4], [5]. Least-squares regression on noisy data is not interpolation, but it faces the same underlying difficulty in a statistical form. A high-degree fit on equispaced points is free to oscillate between the data, it fits the noise as readily as the signal, and the monomial design matrix becomes severely ill-conditioned. Runge’s function therefore provides a demanding test of how model complexity should be selected and controlled.

We study three linear models. Ordinary least squares (OLS) provides the unregularized baseline, whose test error as a function of degree exposes the trade-off directly. Ridge regression [6] and Lasso regression [7] add an ℓ_2 and an ℓ_1 penalty, respectively, trading a controlled increase in bias for a reduction in variance. The Ridge penalty shrinks all coefficients, while the Lasso penalty can set some of them exactly to zero. To estimate the test error and select the degree and penalty, we use two resampling methods: the bootstrap, which we also use to decompose the error into bias and variance, and k -fold cross-validation.

OLS and Ridge have closed-form solutions, whereas Lasso does not, since its penalty is not differentiable at zero. We therefore also solve all three problems iteratively: OLS and Ridge with plain gradient descent, momentum, AdaGrad, RMSprop and Adam [8], full-batch and with mini-batches (stochastic gradient descent), and Lasso with the same update rules applied to a subgradient of its cost. The closed-form OLS and Ridge solutions serve as exact benchmarks for these optimizers. We compute gradients both analytically and by automatic differentiation [9], and verify that the two agree.

All regression models, the gradient-descent solvers, the analytical gradients and the bootstrap are our own implementations. Cross-validation uses the `KFold` and `cross_validate` utilities of `scikit-learn` [10] around our estimators, automatic differentiation uses `JAX` [11], and `scikit-learn`’s Lasso serves as an independent check of our Lasso solver. Because our data are synthetic, we can also compute the true test error of every fitted model from the known f , and we use it to judge how well the resampling estimates select models.

The remainder of this report is structured as follows. Section II presents the models, the bias–variance decomposition, the resampling methods and the optimization methods. Sec-

tion III describes the data, the preprocessing, the code and how it was verified. Section IV presents and discusses the results in the order OLS, Ridge, bias–variance, resampling and model selection, gradient descent, Lasso, and stochastic gradient descent, and section V summarizes our findings and outlines future work. Appendix A declares our use of large language models. All code, and the results behind every figure and table, are available in our repository [12].

II. METHODS

This section presents the models, the error decomposition and the numerical methods of the study: linear regression by ordinary least squares, Ridge and Lasso; the bias–variance decomposition; the bootstrap and k -fold cross-validation; and gradient-based optimization, including its application to Lasso and automatic differentiation. We assume the reader has familiarity with basic linear algebra and statistics.

A. Linear regression

We are given a dataset with n samples $\mathcal{D} = \{(y_i, x_i)\}_{i=0}^{n-1}$ and assume that the targets y_i are generated by

$$\mathbf{y} = \mathbf{f} + \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_n), \quad (2)$$

where $\mathbf{f} = (f(x_0), \dots, f(x_{n-1}))^\top$ collects the values of an unknown function f and the noise is i.i.d. Gaussian with zero mean and variance σ^2 . A polynomial model of degree p is linear in its parameters: with the design matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$, containing the columns $\mathbf{x}, \dots, \mathbf{x}^p$, and the parameter vector $\boldsymbol{\theta} \in \mathbb{R}^p$, the prediction is $\tilde{\mathbf{y}} = \beta_0 + \mathbf{X}\boldsymbol{\theta}$, with the intercept β_0 fit separately from $\boldsymbol{\theta}$ by centering the target rather than through a column of $\mathbf{X}$ (section III-A) [2]. The cost functions below are written for this already-centered target, so β_0 does not appear in them. The goal of linear regression is to choose $\hat{\boldsymbol{\theta}}$ so that the fitted model $g(\mathbf{X}; \hat{\boldsymbol{\theta}}) = \beta_0 + \mathbf{X}\hat{\boldsymbol{\theta}}$ best approximates the unknown function f on the data $\mathcal{D}$ [13].

1) *Singular value decomposition*: Any matrix $\mathbf{X}$ can be factorized as $\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$, with $\mathbf{U}$ and $\mathbf{V}$ orthogonal and $\boldsymbol{\Sigma}$ rectangular diagonal with the singular values $\sigma_0 \geq \sigma_1 \geq \dots \geq 0$. The ratio $\kappa_2(\mathbf{X}) = \sigma_{\max}/\sigma_{\min}$ is the 2-norm condition number of $\mathbf{X}$ [2]. The monomial columns $\mathbf{x}^j$ on $[-1, 1]$ become nearly parallel as j grows, so $\kappa_2(\mathbf{X})$ grows exponentially with the degree; this ill-conditioning is behind several of our numerical results.

2) *Ordinary least squares*: Ordinary least squares (OLS) chooses $\boldsymbol{\theta}$ to minimize the mean squared error [13],

$$C_{\text{OLS}}(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2. \quad (3)$$

The cost is convex, since its Hessian $\mathbf{H} = \frac{2}{n} \mathbf{X}^\top \mathbf{X}$ is positive semi-definite, so every stationary point is a global minimizer. Setting the gradient $\nabla_{\boldsymbol{\theta}} C_{\text{OLS}} = \frac{2}{n} \mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$ to zero gives the normal equations; if $\mathbf{X}$ has full column rank, the normal equations have the unique solution

$$\hat{\boldsymbol{\theta}}_{\text{OLS}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (4)$$

If $\mathbf{X}$ is rank-deficient, $\mathbf{X}^\top \mathbf{X}$ is singular and the OLS minimizer is not unique [13]. In this case, the Moore–Penrose pseudoinverse gives the minimum-norm OLS solution,

$$\hat{\boldsymbol{\theta}}_{\text{OLS}} = \mathbf{X}^+ \mathbf{y} = \mathbf{V}\boldsymbol{\Sigma}^+ \mathbf{U}^\top, \quad (5)$$

where $\mathbf{X}^+$ denotes the Moore–Penrose pseudoinverse [14]. We compute OLS this way as forming $\mathbf{X}^\top \mathbf{X}$ squares the condition number, which for our design matrices can exhaust double precision at moderate degree [2].

3) *Ridge regression*: Ridge regression [6] adds the ℓ_2 -norm of the parameter vector to the as a penalty to the cost function,

$$C_{\text{Ridge}}(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2, \quad \lambda \geq 0, \quad (6)$$

where the penalty is added to the same $1/n$ -normalized cost used throughout; this is the convention of our code, and it fixes the meaning of λ in all results. This yields the minimizer

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}}(\lambda) = (\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (7)$$

For $\lambda > 0$ the Hessian $\frac{2}{n} \mathbf{X}^\top \mathbf{X} + 2\lambda \mathbf{I}$ is positive definite, so the cost is strictly convex and the minimizer unique for any $\mathbf{X}$, full rank or not [2]. In terms of the SVD, the Ridge prediction is

$$\tilde{\mathbf{y}}_{\text{Ridge}} = \sum_j d_j \mathbf{u}_j \mathbf{u}_j^\top \mathbf{y}, \quad d_j = \frac{\sigma_j^2}{\sigma_j^2 + n\lambda}, \quad (8)$$

with $d_j \in [0, 1]$ for $\lambda > 0$, whereas OLS has $d_j = 1$ for every $\sigma_j > 0$. Directions with $\sigma_j^2 \ll n\lambda$ are suppressed, directions with $\sigma_j^2 \gg n\lambda$ are kept as in OLS. Ridge therefore suppresses precisely the directions associated with small singular values, which make the OLS solution sensitive to noise [2].

4) *Lasso regression*: Lasso [7] applies an ℓ_1 penalty,

$$C_{\text{Lasso}}(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1. \quad (9)$$

The cost is convex but not differentiable where a coefficient is zero, and it has no closed-form minimizer [13]. Writing $\mathbf{r} = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}}$ for the residual and $\mathbf{x}_j$ for column j of $\mathbf{X}$, the minimizer satisfies

$$\frac{2}{n} \mathbf{x}_j^\top \mathbf{r} = \lambda \operatorname{sgn}(\hat{\theta}_j) \quad \text{if } \hat{\theta}_j \neq 0, \quad \left| \frac{2}{n} \mathbf{x}_j^\top \mathbf{r} \right| \leq \lambda \quad \text{if } \hat{\theta}_j = 0. \quad (10)$$

A coefficient therefore stays exactly zero as long as its column’s correlation with the residual is too weak to outweigh the penalty λ ; it only enters the model once that correlation exceeds λ . Ridge’s penalty is smooth at zero, so it has no such dead zone: any nonzero correlation pulls the coefficient away from zero, just scaled down. This is why Lasso performs variable selection while Ridge only shrinks [2, Prop. 3.7].

5) *Error measures*: We evaluate a prediction $\tilde{\mathbf{y}}$ of targets $\mathbf{y}$ by

$$\text{MSE} = \frac{1}{n} \sum_i (y_i - \tilde{y}_i)^2, \quad R^2 = 1 - \frac{\sum_i (y_i - \tilde{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \quad (11)$$

with $\bar{y}$ the mean of $\mathbf{y}$. An unbiased model with no variance reaches $\text{MSE} = \sigma^2$ in expectation; R^2 compares against predicting $\bar{y}$ and is negative for a model that does worse than that constant [2].

B. Bias–variance decomposition

We decompose the expected squared error $\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2]$, the quantity MSE estimates, into bias, variance, and irreducible noise. The prediction $\tilde{\mathbf{y}}$ at a fixed set of test inputs is a random quantity, because the parameters are fitted on a training set that is itself random. Below, $\mathbb{E}$ is taken jointly over the draw of the training set (in the bootstrap: over the resampled training sets) and over the noise ε on the test targets, which is independent of the training set [2]. Squares of vectors are elementwise and averaged over the test points.

Writing $\mathbf{y} = \mathbf{f} + \varepsilon$ and adding and subtracting $\mathbb{E}[\tilde{\mathbf{y}}]$,

$$\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] = \mathbb{E} \left[\underbrace{(\mathbf{f} - \mathbb{E}[\tilde{\mathbf{y}}])}_{\mathbf{a}} + \underbrace{(\mathbb{E}[\tilde{\mathbf{y}}] - \tilde{\mathbf{y}})}_{\mathbf{b}} + \underbrace{\varepsilon}_{\mathbf{c}} \right]^2. \quad (12)$$

The cross terms vanish: $\mathbf{a}$ is deterministic and $\mathbb{E}[\mathbf{b}] = \mathbf{0}$, so $\mathbb{E}[\mathbf{ab}] = \mathbf{a} \mathbb{E}[\mathbf{b}] = \mathbf{0}$; $\mathbb{E}[\mathbf{ac}] = \mathbf{a} \mathbb{E}[\varepsilon] = \mathbf{0}$; and ε is independent of $\tilde{\mathbf{y}}$, so $\mathbb{E}[\mathbf{bc}] = \mathbb{E}[\mathbf{b}] \mathbb{E}[\varepsilon] = \mathbf{0}$. What remains is

$$\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] = \underbrace{(\mathbf{f} - \mathbb{E}[\tilde{\mathbf{y}}])^2}_{\text{Bias}^2[\tilde{\mathbf{y}}]} + \underbrace{\mathbb{E}[(\tilde{\mathbf{y}} - \mathbb{E}[\tilde{\mathbf{y}}])^2]}_{\text{Var}[\tilde{\mathbf{y}}]} + \sigma^2. \quad (13)$$

Averaged over m test points,

$$\text{Bias}^2[\tilde{\mathbf{y}}] = \frac{1}{m} \sum_i (f_i - \mathbb{E}[\tilde{y}_i])^2, \quad (14)$$

$$\text{Var}[\tilde{\mathbf{y}}] = \frac{1}{m} \sum_i \mathbb{E}[(\tilde{y}_i - \mathbb{E}[\tilde{y}_i])^2], \quad (15)$$

where we estimate $\mathbb{E}[\tilde{y}_i]$ and the inner expectation of eq. (15) by averages over the bootstrap replicates.

The bias measures how far the average prediction is from f ; it is large when the model is too rigid to represent f . The variance measures how much the prediction fluctuates when the training set changes; it grows with model complexity. The noise variance σ^2 is irreducible and is a floor for the expected test error [2].

Since f is unknown in practice, the bias is usually measured against the observed targets $\mathbf{y} = \mathbf{f} + \varepsilon$. Because ε has mean zero and is independent of $\tilde{\mathbf{y}}$,

$$\frac{1}{m} \sum_i \mathbb{E}[(y_i - \mathbb{E}[\tilde{y}_i])^2] = \text{Bias}^2[\tilde{\mathbf{y}}] + \sigma^2, \quad (16)$$

so the measured bias absorbs the noise variance, and the measured $\text{Bias}^2 + \text{Var}$ adds up to the full test MSE. Since our data are synthetic, we report both versions.

C. Resampling methods

A single train–test split gives one realization of the test error; its variance can be large when the test set is small. Resampling methods reuse the available data to estimate the expected test error of a model, and to choose between models, without collecting new data.

1) *Bootstrap*: The bootstrap [15] approximates the distribution of the training sets by the empirical distribution of the one training set we have: we draw B training sets of the same size n_{tr} with replacement, refit the model on each, and predict on a fixed test set. The spread of the B predictions estimates $\text{Var}[\tilde{\mathbf{y}}]$, their mean estimates $\mathbb{E}[\tilde{\mathbf{y}}]$ in eq. (14), and the average squared error estimates the test MSE. A bootstrap sample contains on average only a fraction $1 - (1 - 1/n_{\text{tr}})^{n_{\text{tr}}} \approx 1 - e^{-1} \approx 0.632$ of the distinct training points, so every bootstrap fit is made from effectively fewer points than the fit on the full training set [3].

2) *k-fold cross-validation*: In k -fold cross-validation (CV), the data are split into k disjoint folds of (almost) equal size. The model is trained k times, each time on $k - 1$ folds, and evaluated on the held-out fold [16]. The CV estimate of the test error is the mean of the k fold errors,

$$\text{CV}_k = \frac{1}{k} \sum_{i=1}^k \text{MSE}(\mathbf{y}_i, \tilde{\mathbf{y}}_i^{(-i)}), \quad (17)$$

where $\tilde{\mathbf{y}}_i^{(-i)}$ is the prediction for fold i of the model trained without it. We select hyperparameters (the degree p and λ) by minimizing CV_k over a grid [2]. Larger k trains on more data per fit ($1 - 1/k$ of it), which reduces the pessimistic bias of the estimate; but the k training sets then overlap more, so the fold errors become more correlated and the average has higher variance [3].

D. Gradient-based optimization

1) *Gradient descent*: For a differentiable cost $C(\boldsymbol{\theta})$, gradient descent (GD) iterates

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \gamma \nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}^{(k)}), \quad (18)$$

with learning rate $\gamma > 0$. Newton’s method, $\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \mathbf{H}^{-1} \nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}^{(k)})$, would solve a quadratic cost in one step, but forming and inverting the $p \times p$ Hessian costs $\mathcal{O}(p^3)$ for p parameters; GD replaces $\mathbf{H}^{-1}$ by $\gamma \mathbf{I}$ and costs one gradient evaluation per iteration [2].

For the quadratic OLS and Ridge costs the analysis is exact. The error $\mathbf{e}^{(k)} = \boldsymbol{\theta}^{(k)} - \hat{\boldsymbol{\theta}}$ obeys $\mathbf{e}^{(k+1)} = (\mathbf{I} - \gamma \mathbf{H}) \mathbf{e}^{(k)}$, so along the eigenvector of $\mathbf{H}$ with eigenvalue λ_i the error is multiplied by $1 - \gamma \lambda_i$ in each step. The iteration converges from any starting point if and only if $|1 - \gamma \lambda_i| < 1$ for all i with $\lambda_i > 0$, that is

$$0 < \gamma < \frac{2}{\lambda_{\max}(\mathbf{H})}. \quad (19)$$

The slowest direction contracts by $1 - \gamma \lambda_{\min}$, so with $\gamma = 1/\lambda_{\max}$ reaching a relative accuracy ϵ takes about $\kappa(\mathbf{H}) \ln(1/\epsilon)$ iterations, where $\kappa(\mathbf{H}) = \lambda_{\max}/\lambda_{\min} = \kappa(\mathbf{X})^2$ for OLS. For Ridge with centered features, an unpenalized intercept and $\lambda \leq 1$, $\lambda_{\min} \geq 2\lambda$, so $\kappa(\mathbf{H}) \leq \lambda_{\max}/(2\lambda)$ however ill-conditioned $\mathbf{X}$ is.

2) *Momentum*: Momentum adds a fraction of the previous step,

$$\mathbf{v}^{(k)} = \beta \mathbf{v}^{(k-1)} + \gamma \nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}^{(k)}), \quad \boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \mathbf{v}^{(k)}, \quad (20)$$

with $\beta \in [0, 1)$ and $\mathbf{v}^{(0)} = \mathbf{0}$. Along directions where the gradient keeps its sign the steps accumulate, while oscillations across a narrow valley partly cancel; this is what makes momentum effective for ill-conditioned Hessians [2], [14].

3) *Adaptive learning rates:* AdaGrad [17] scales every coordinate by the accumulated squared gradients,

$$\mathbf{r}^{(k)} = \mathbf{r}^{(k-1)} + \mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}, \quad (21)$$

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \frac{\gamma}{\sqrt{\mathbf{r}^{(k)}} + \epsilon} \odot \mathbf{g}^{(k)}, \quad (22)$$

where $\mathbf{g}^{(k)}$ is the gradient, $\odot$ the elementwise product, and the square root and division are elementwise. The effective step of each coordinate decreases monotonically. RMSprop replaces the sum by an exponentially decaying average, $\mathbf{v}^{(k)} = \rho \mathbf{v}^{(k-1)} + (1 - \rho) \mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}$, so the step size can recover [14]. Adam [8] combines a decaying average of the gradient (momentum) with RMSprop's scaling and corrects both for their initialization at zero:

$$\mathbf{m}^{(k)} = \beta_1 \mathbf{m}^{(k-1)} + (1 - \beta_1) \mathbf{g}^{(k)}, \quad (23)$$

$$\mathbf{v}^{(k)} = \beta_2 \mathbf{v}^{(k-1)} + (1 - \beta_2) \mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}, \quad (24)$$

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \gamma \frac{\mathbf{m}^{(k)} / (1 - \beta_1^k)}{\sqrt{\mathbf{v}^{(k)} / (1 - \beta_2^k)} + \epsilon}. \quad (25)$$

We use $\beta = 0.9$ for momentum, $\rho = 0.9$ for RMSprop, $\beta_1 = 0.9$, $\beta_2 = 0.999$ for Adam, and $\epsilon = 10^{-8}$. Because Adam divides by the running root-mean-square gradient, its step does not shrink in proportion to the gradient near the minimum; with a fixed γ it can keep taking steps of order γ there.

4) *Stochastic gradient descent:* For a cost that is an average over the data, $C(\boldsymbol{\theta}) = \frac{1}{n} \sum_i c_i(\boldsymbol{\theta})$, stochastic gradient descent (SGD) replaces the full gradient by the average over a mini-batch $\mathcal{B}_k$ of M points,

$$\mathbf{g}^{(k)} = \frac{1}{M} \sum_{i \in \mathcal{B}_k} \nabla_{\boldsymbol{\theta}} c_i(\boldsymbol{\theta}^{(k)}), \quad (26)$$

an unbiased estimate of the full gradient [14]. We shuffle the training set once per epoch and split it into $\lceil n_{\text{tr}}/M \rceil$ mini-batches, so one epoch costs the same number of floating-point operations as one full-batch step but makes $\lceil n_{\text{tr}}/M \rceil$ updates. The gradient noise does not vanish at the minimum, so with a fixed learning rate the iterates fluctuate around it; a decaying learning rate, here $\gamma_t = \gamma_0 / (1 + \delta t)$ with t the number of updates, damps these fluctuations [2]. Any of the update rules above can be applied to $\mathbf{g}^{(k)}$.

5) *Gradient descent for Lasso:* Since eq. (9) is not differentiable at $\theta_j = 0$, its gradient is not defined there. The *subgradient method* replaces the derivative of $|\theta_j|$ by an element of the subdifferential $\partial|\theta_j|$, i.e. it uses

$$\mathbf{g}(\boldsymbol{\theta}) = \frac{2}{n} \mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) + \lambda \text{sign}(\boldsymbol{\theta}), \quad (27)$$

with $\text{sign}(0) = 0$ (the minimal-norm choice) and no penalty on the intercept, in place of the gradient in any of the update rules above. Unlike GD on a smooth cost, the subgradient method with a fixed step does not converge to the minimizer, only to a neighborhood of it whose size is proportional to the step: every nonzero coefficient receives a push of magnitude $\gamma\lambda$ towards

zero in each step, so a coefficient whose optimal value is zero oscillates around zero instead of reaching it. Exact zeros, the defining property of the Lasso solution, therefore appear only by coincidence.

6) *Automatic differentiation:* Automatic differentiation (AD) computes derivatives of a function given as a program by applying the chain rule to the sequence of elementary operations the program executes [9]. It is neither symbolic differentiation, which manipulates expressions and can produce exponentially large derivative expressions, nor numerical differentiation, whose finite differences carry truncation and round-off errors: AD evaluates the derivative exactly up to floating-point round-off, at a specific input. In reverse mode, which JAX's `grad` uses for a scalar cost, the program is run forward once while storing intermediate values, and derivatives are then propagated backwards; the full gradient costs a small constant multiple of one evaluation of the cost, independent of the number of parameters [9]. At a point where the function is not differentiable, AD returns the derivative of whichever branch the program takes; for $|\theta|$ at $\theta = 0$, JAX returns 1, which is an element of $\partial|\theta| = [-1, 1]$ and hence a valid subgradient, although not the minimal-norm one (0).

III. IMPLEMENTATION

All methods are implemented in a Python package; the source code, the tests, the experiment scripts and the results behind every number in this report are in the project repository [12]. This section describes the data, the preprocessing, the structure of the code, and how we verified it.

A. Data and preprocessing

We draw $n = 100$ inputs uniformly from $[-1, 1]$, evaluate Runge's function eq. (1), and add i.i.d. Gaussian noise with $\sigma = 0.1$ (eq. (2), fig. 1). The data are split 80/20 into a training and a test set. Unless stated otherwise, all results use this reference data set (seed 42). The statistics in section IV-D repeat the whole pipeline on 31 independent data sets (seeds 0–29 and 42). All settings are defined in one configuration file, from which every figure and table is regenerated by a single command (`make figures`).

The design matrix has the columns $\mathbf{x}, \dots, \mathbf{x}^p$; it carries no all-ones intercept column. We standardize every column to zero mean and unit variance, using means and standard deviations computed on the training rows only (scikit-learn's `StandardScaler` [10]), and apply the same transformation to the test rows. Inside cross-validation, the scaler is part of a `Pipeline`, so it is refitted on the training folds of every split. We scale for three reasons. First, the raw columns $\mathbf{x}^j$ have very different magnitudes on $[-1, 1]$, and the penalties of Ridge and Lasso would then shrink the coefficients of the small columns much more than those of the large ones; after standardization, λ acts on all coefficients on the same footing [2], [16]. Second, standardization improves the conditioning of $\mathbf{X}^\top \mathbf{X}$, which sets the speed of gradient descent (eq. (19)). Third, every model fits its intercept by centering $\mathbf{y}$ on its training mean instead of through a column of $\mathbf{X}$: since the standardized columns are themselves centered, this

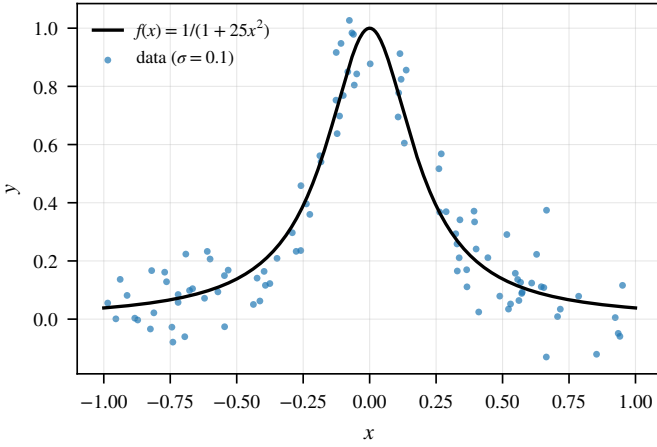


Fig. 1. The reference data set: $n = 100$ points $x_i \sim \mathcal{U}(-1, 1)$ with $y_i = f(x_i) + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, 0.1^2)$, and Runge’s function f .

gives exactly the coefficients a joint fit with a free, unpenalized intercept would, and that intercept is exactly the mean of the training targets. OLS predictions do not depend on the scaling (it is an invertible linear reparametrization of the same model); Ridge, Lasso and every gradient method do. The target itself is never scaled, only centered internally by the model.

B. Code structure

The package `fys_stk4155_p1` contains four subpackages: `data` (sampling of Runge’s function and the design matrix), `regression` (the models, costs and gradients), `optimization` (the update rules and learning-rate schedules), and `resampling` (the bootstrap and k -fold CV). Thin scripts combine them into the experiments and figures.

All models inherit from a common estimator class that follows the scikit-learn convention (hyperparameters in the constructor, `fit` and `predict`), so they can be used directly in scikit-learn’s `Pipeline`, `KFold` and `cross_validate`. OLS solves the least-squares problem through the SVD-based pseudoinverse with a relative singular-value cutoff of machine precision times $\max(n, p)$. Ridge solves the normal equations eq. (7) with `numpy.linalg.solve`; the penalty makes the system well-posed for $\lambda > 0$. `GradientDescent` minimizes the OLS/Ridge (or Lasso) cost with any of the update rules plain, momentum, AdaGrad, RMSprop and Adam, full-batch or with mini-batches, using either the analytical gradient or the gradient from automatic differentiation with JAX [11] (in double precision). Lasso minimizes the Lasso cost with the same code, using the subgradient eq. (27) in place of the gradient; unless stated otherwise we use Adam with $\gamma = 0.05$ and 5000 iterations. All update rules follow algorithm 1, with line 3 replaced by their own update.

The bootstrap is our own code. Cross-validation uses scikit-learn’s `KFold` for the fold assignment and `cross_validate` for the loop over folds, with our estimators inside a pipeline. Every function that involves randomness takes an explicit seed, and all numbers in this report are regenerated deterministically by `make experiments`.

Algorithm 1 Basic gradient descent

Require: Initial parameters $\theta^{(0)}$, learning rate $\gamma > 0$, tolerance $\varepsilon > 0$, maximum iterations $N_{\max}$
Ensure: Approximate minimizer θ^* of $C(\theta)$

```

1:  $k \leftarrow 0$ 
2: while  $\|\nabla C(\theta^{(k)})\|_{\infty} > \varepsilon$  and  $k < N_{\max}$  do
3:    $\theta^{(k+1)} \leftarrow \theta^{(k)} - \gamma \nabla C(\theta^{(k)})$  ▷ Equation (18)
4:    $k \leftarrow k + 1$ 
5: end while
6: return  $\theta^{(k)}$ 

```

C. Verification

The package has a test suite of 164 unit tests, run by continuous integration on every push. The tests most relevant for trusting the results check the code against closed-form results and independent implementations:

- **OLS and Ridge:** OLS recovers the coefficients of an exactly linear data set; Ridge with $\lambda = 0$ equals OLS; the Ridge coefficients shrink monotonically in λ ; and the intercept is not penalized.
- **Gradients:** the analytical gradients agree with those from JAX to a maximum relative difference of 1.0×10^{-15} over 100 random θ at degrees 5, 10 and 15, for OLS, Ridge and Lasso away from $\theta_j = 0$. At $\theta_j = 0$ they differ by exactly λ per coefficient, since `numpy.sign(0) = 0` while JAX differentiates $|\theta|$ to 1 at the origin.
- **Gradient descent vs. closed form:** with $\gamma = 1/L$, plain GD converges linearly to the closed-form OLS and Ridge solutions (fig. 12), and the stability boundary eq. (19) is reproduced (fig. 13).
- **Optimizers:** each update rule reproduces its formula for one step on known inputs, and converges on a quadratic test problem.
- **Lasso:** on a well-conditioned test problem, our subgradient solver agrees with scikit-learn’s coordinate-descent Lasso (with $\alpha = \lambda/2$) to 0.05 in every coefficient; the number of small coefficients grows with λ ; and the intercept is not penalized.
- **Resampling:** the bootstrap and CV are deterministic for a given seed, and the bias measured against $\mathbf{y}$ exceeds the bias measured against $\mathbf{f}$ by approximately σ^2 (eq. (16)).

IV. RESULTS AND DISCUSSION

Unless stated otherwise, all results use the reference data set of section III-A: $n = 100$, $\sigma = 0.1$ (so $\sigma^2 = 0.01$ is the floor of the expected test MSE), an 80/20 split with seed 42, and features standardized on the training rows. Polynomial degrees run up to $p = 20$.

A. Ordinary least squares

Figure 2 shows the training and test error of OLS as a function of the degree. The training MSE decreases monotonically, as it must for nested least-squares models, from 0.10 at $p = 1$ to 0.0078 at $p = 20$, below σ^2 : the high-degree fits begin to follow the noise. The test MSE falls steeply up to $p \approx 8$, where the polynomial is first flexible enough to represent the narrow peak of f at $x = 0$, is smallest at $p = 12$ (0.0086, $R^2 = 0.63$),

and then rises slowly to 0.0105 at $p = 20$. This is the shape of fig. 2.11 of Hastie et al. [3], but with a much flatter right branch than one might expect from Runge's phenomenon. That the minimum lies slightly below σ^2 is not a contradiction: with 20 test points the test MSE has a standard error of roughly $\sigma^2 \sqrt{2/20} \approx 0.3 \sigma^2$, which also makes the location of the minimum between $p \approx 10$ and $p = 14$ essentially arbitrary. The low R^2 reflects that most of the variance of y in the test set is noise; the negative R^2 at low degree occurs because the 20 test points happen to contain few points near the peak, so their variance is small.

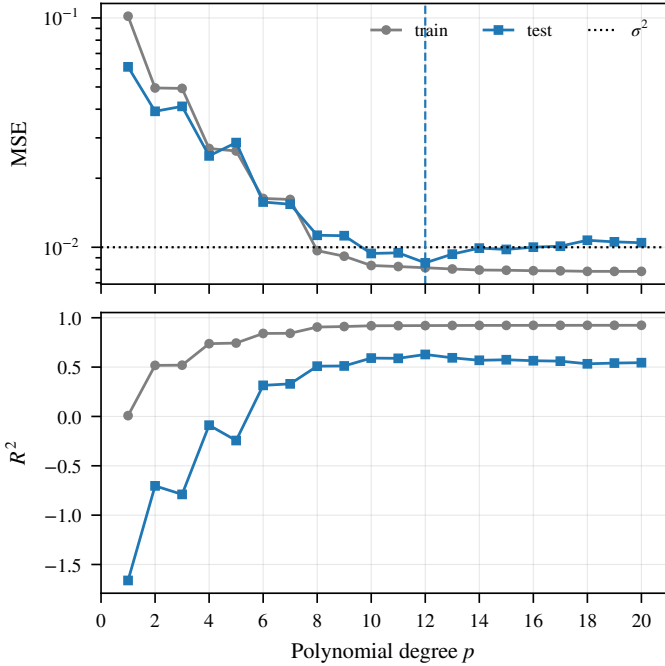


Fig. 2. OLS training and test MSE (top) and R^2 (bottom) vs. polynomial degree for the reference data set ($n = 100$, $\sigma = 0.1$, 80/20 split). The dotted line is σ^2 ; the dashed line marks the minimum test MSE ($p = 12$).

The coefficients, shown in fig. 3, tell the other half of the story. Up to $p \approx 6$ they are of order one, but from there on they grow by roughly a factor of three every two degrees, to $\max_j |\hat{\theta}_j| = 3.3 \times 10^3$ at $p = 20$, with alternating signs between neighboring powers. The fitted curve is a near-cancellation of large, nearly collinear terms: the condition number of the standardized design matrix grows from 25 at $p = 5$ to 1.7×10^3 at $p = 10$ and 1.6×10^7 at $p = 20$. Such coefficients are determined by the particular noise realization, which is the variance that the bias–variance analysis below quantifies.

With more data, the minimum moves to higher degree and the high-degree deterioration disappears (fig. 4): for $n = 400$ the test MSE is flat at $\approx \sigma^2$ from $p \approx 12$ to 20, whereas for $n = 50$ (with only 10 test points) the curve is erratic beyond $p = 12$ and reaches 3.2 at $p = 20$. The erratic curves for $n = 50$ and $n = 200$ are dominated by single test points near the ends of the interval, a mechanism we return to in section IV-D.

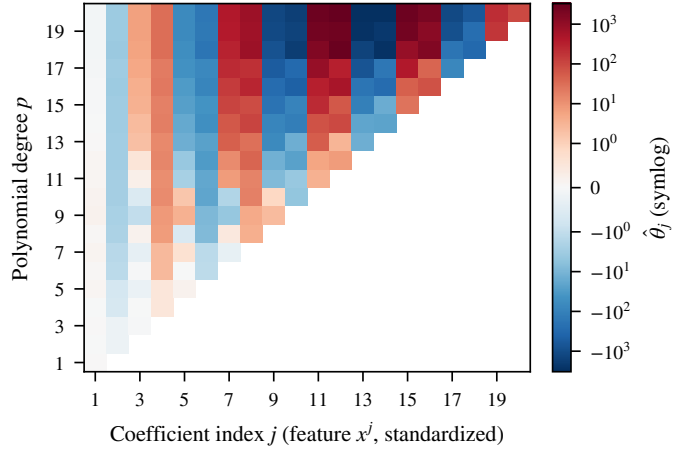


Fig. 3. OLS coefficients $\hat{\theta}_j$ of the standardized feature x^j (column) for each degree p (row), on a symmetric logarithmic color scale. The intercept, equal to the mean training target, is not shown.

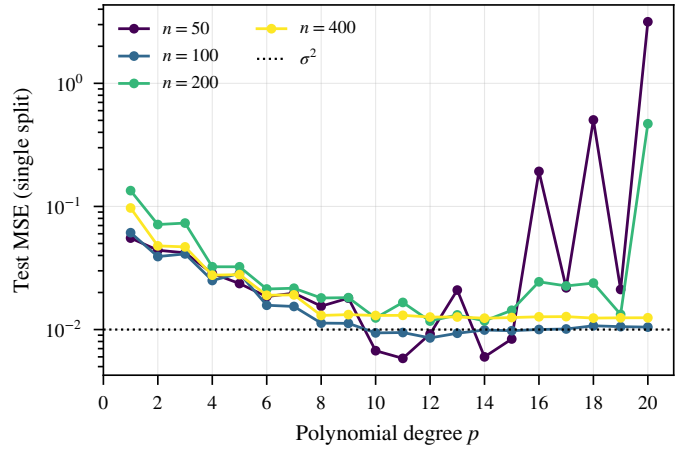


Fig. 4. OLS test MSE vs. degree for $n \in \{50, 100, 200, 400\}$ points ($\sigma = 0.1$, 80/20 split, seed 42 for every n).

B. Ridge regression

Figure 5 shows the mechanism of Ridge regression at $p = 15$. The squared singular values of the standardized design matrix, σ_i^2/n , span ten orders of magnitude, from 9.8 to 3.8×10^{-10} . According to eq. (8), a penalty λ suppresses the modes with $\sigma_i^2/n \lesssim \lambda$ and leaves the others untouched, so increasing λ by a decade removes roughly one more mode. $\lambda = 10^{-10}$ is essentially OLS; $\lambda = 1$ keeps only the first three modes. The ridge trace (fig. 6) shows the corresponding coefficients: the large, alternating OLS coefficients collapse between $\lambda \approx 10^{-9}$ and 10^{-6} , as the smallest modes are removed.

Figure 7 repeats the analysis of fig. 2 for Ridge. A small penalty ($\lambda = 10^{-8}$) reproduces the OLS curve up to $p \approx 14$ and removes the slight rise beyond it (0.0096 instead of 0.0105 at $p = 20$); $\lambda = 10^{-6}$ is similar. Larger penalties make the curve flat in p , because the extra degrees only add modes that the penalty suppresses, but at a higher level: 0.013 for $\lambda = 10^{-4}$ and 0.023 for $\lambda = 10^{-2}$. Here the penalty removes modes that are needed to represent the peak of f , and the bias

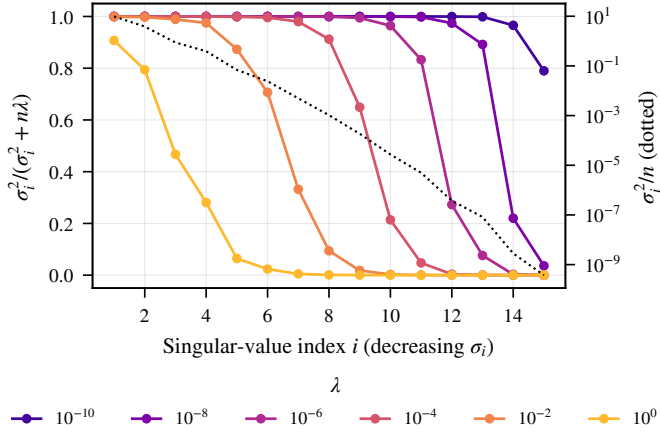


Fig. 5. Ridge shrinkage factors $d_i = \sigma_i^2 / (\sigma_i^2 + n\lambda)$ (eq. (8)) of the SVD modes of the standardized design matrix at $p = 15$ (training set of the reference data), for λ from 10^{-10} to 1. Dotted, right axis: σ_i^2/n .

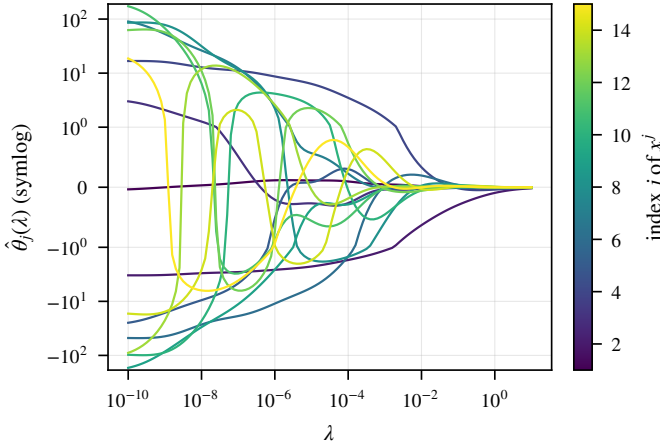


Fig. 6. Ridge trace at $p = 15$: coefficients $\hat{\theta}_j(\lambda)$ of the standardized features, colored by the power j .

dominates. On this data set Ridge therefore makes the choice of degree less critical, but it does not lower the best achievable test error; we test this more carefully in section IV-D.

C. Bias–variance trade-off

Figure 8 shows the bootstrap decomposition (eq. (13)) for OLS with $B = 100$ resamples of the training set, evaluated on the fixed test set. At low degree the error is dominated by the bias: at $p = 2$ the squared bias against f is 0.032 and the variance 0.002. Increasing the degree trades bias for variance, and the bootstrap test error is smallest at $p = 6$ (0.021), where $\text{Bias}^2 = 0.009$ and $\text{Var} = 0.006$ are of the same size. Beyond it the variance explodes, from 0.006 at $p = 6$ to 0.57 at $p = 10$ and 15 at $p = 15$: a factor of 2.6×10^3 between $p = 6$ and $p = 15$. The two bias estimates differ by approximately σ^2 , as eq. (16) predicts (e.g. 0.015 against y and 0.009 against f at $p = 6$; the difference fluctuates around $\sigma^2 = 0.01$ since only 20 test points enter the average).

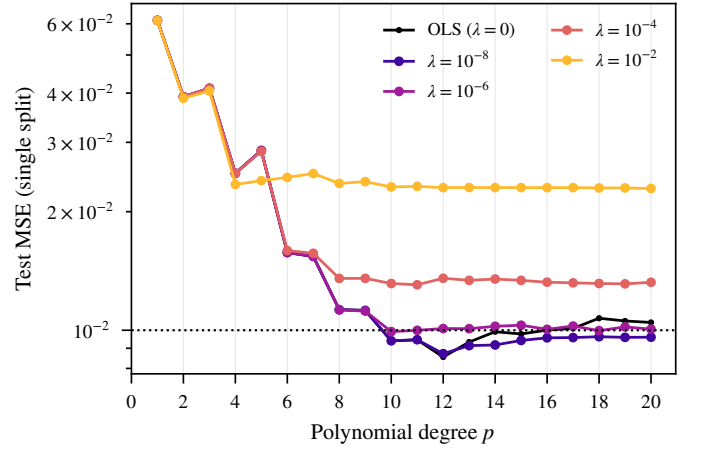


Fig. 7. Ridge test MSE vs. degree for four values of λ , compared with OLS (reference data set, single split). Dotted: σ^2 .

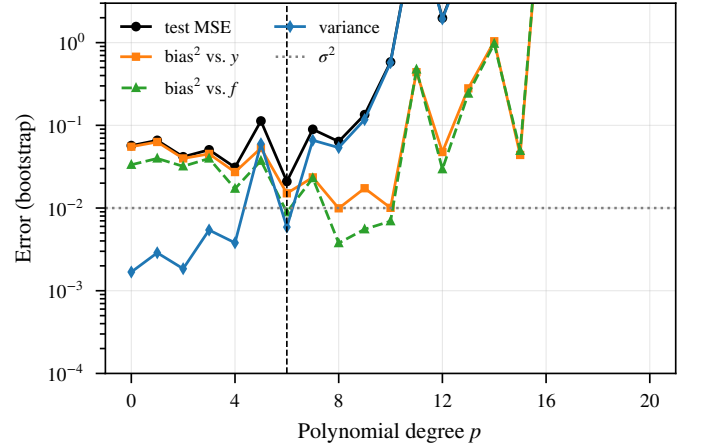


Fig. 8. Bootstrap bias–variance decomposition of the OLS test error ($B = 100$, reference data set). The squared bias is measured both against the noisy test targets y (eq. (16), absorbs σ^2) and against the true f (eq. (14)). Dashed: minimum of the bootstrap test MSE ($p = 6$). Values above 3 are cut off.

Two features of the figure deserve comment. First, the bootstrap error is minimized at $p = 6$, whereas the single test split of fig. 2 favors $p = 12$; both use the same training and test points. The difference arises because each bootstrap fit sees only about 63% of the distinct training points (section II-C) and is therefore much less stable at high degree than the fit on all 80 points. The bootstrap thus estimates the variance of a model trained on fewer, and duplicated, points. Second, the bias is not monotone in p beyond $p \approx 8$: the average over bootstrap fits is itself contaminated by a few fits that go wild near the ends of the interval, so $\mathbb{E}[\hat{y}]$ is not a smooth polynomial fit either. We examine the origin of these wild fits in section IV-D.

The decomposition responds to n and σ as expected (figs. 19 and 20 in appendix B). With more data the variance at a given degree drops while the bias hardly changes, so the minimum moves from $p = 4$ at $n = 50$ to 6, 10 and 12 at $n = 100, 200$ and 400. More noise raises the floor and the variance, and the

optimal degree falls from 6 ($\sigma \leq 0.2$) to 4 ($\sigma = 0.4$).

D. Resampling and model selection

1) *Which degree do the estimators select?:* Figure 9(a) compares four estimates of the OLS test error on the reference data set, all computed from the 80 training points: the bootstrap (on the held-out 20 points), 5- and 10-fold CV, and the single split. Since our data are synthetic, we can also compute the *true* expected test error of the model fitted on all 80 points, $\mathbb{E}_x[(\hat{f}(x) - f(x))^2] + \sigma^2$ for $x \sim \mathcal{U}(-1, 1)$, on a dense grid. The true error is smallest at $p = 14$ (0.0133) and stays below 0.026 up to $p = 20$. The four estimates agree up to $p \approx 7$, but then separate: the bootstrap selects $p = 6$, both CV variants $p = 8$ and the single split $p = 12$, and the bootstrap and CV estimates grow by several orders of magnitude at high degree, where the true error does not.

Figure 10 shows where the large estimates come from. In the 5-fold split that dominates the CV estimate at $p = 15$, the training folds happen to contain no point beyond $x = 0.854$, while the held-out fold contains the largest training input, $x = 0.951$. The degree-15 polynomial fitted on the other folds must extrapolate to that point and predicts -34.8 there, while the fit on all 80 points is well-behaved. This single point contributes 87% of the squared error of that fold and makes the fold MSE 88. The mechanism is the statistical counterpart of Runge’s phenomenon: a high-degree polynomial is well constrained only where it has data, and it diverges quickly just outside that range. It affects the bootstrap even more: for the reference data, 36% of the bootstrap samples leave at least one test point outside their x -range. The resampling estimates thus include extrapolation errors that the final model, trained on all points, does not make. Leave-out-based estimators are therefore pessimistic for high-degree polynomials, and increasingly so the fewer points each fit uses (bootstrap > 5 -fold > 10 -fold).

Figure 9(b) shows that this is systematic. Over 31 data sets, the median selected degree is 8 for the bootstrap, 10 for 5-fold and 12 for 10-fold CV, and 13 for the single split, while the degree with the smallest true error has median 14. The selection matters less than the spread suggests, because the true error is flat over a wide range of degrees: the mean true error of the model at the selected degree is 0.0179 ± 0.0008 for the bootstrap, 0.024 ± 0.006 for 5-fold CV, 0.028 ± 0.008 for 10-fold CV and 0.022 ± 0.006 for the single split (mean $\pm$ standard error over the 31 data sets), against 0.0131 ± 0.0003 at the best possible degree. The bootstrap, although it selects too low a degree, is the most *reliable* selector: CV and the single split occasionally select a degree whose fit extrapolates badly at the ends of the interval, which is what produces their large standard errors. The two resampling methods therefore do not point to the same model complexity, and neither points to the best one, for a reason specific to polynomials: their error estimates are dominated by extrapolation.

2) *Ridge and Lasso over degree and penalty:* Figure 11 shows the CV error of Ridge and Lasso over the grid of degrees and penalties on the reference training set. The Ridge surfaces have a valley running diagonally: higher degrees need

larger penalties. In their lower right corner, small λ at high degree, the CV error explodes as for OLS, because the penalty is too weak to prevent the extrapolation failures of fig. 10; $\lambda \gtrsim 10^{-7}$ suppresses them. 5-fold CV selects Ridge with $p = 11$, $\lambda = 10^{-6}$ (CV MSE 0.0121) and 10-fold CV $p = 12$, $\lambda = 10^{-5.5}$ (0.0104), whereas OLS with 5-fold CV selects $p = 8$ with a CV MSE of 0.0285. The Ridge minimum is less than half the OLS minimum, but on the same five folds the difference is -0.016 ± 0.013 (mean $\pm$ standard error over folds), not significant: Ridge mainly avoids the one fold with the extrapolation failure that inflates the OLS estimate. The Lasso surface looks different. It hardly depends on the degree beyond $p \approx 8$ and has its minimum at a low degree and a large penalty, $p = 6$ and $\lambda = 10^{-2.5}$ (CV MSE 0.0222). As section IV-F shows, this reflects our solver more than the Lasso model: with 5000 subgradient steps, the high-degree fits are far from the Lasso solution.

E. Gradient descent

Linear regression is an ideal test case for gradient descent: the closed-form solutions are exact benchmarks, the gradient is known analytically, and the convexity of the cost guarantees convergence for a small enough learning rate. We compare the optimizers at $p = 5$, where plain GD converges in a reasonable number of iterations, and use $p = 10$ and 15 to study ill-conditioning. All runs start from $\theta = 0$.

Figure 12 shows plain GD with $\gamma = 1/L$ at $p = 5$, where $\kappa(H) = 604$ for OLS and 510 for Ridge with $\lambda = 10^{-3}$. The relative cost gap decreases linearly on the logarithmic scale, as the eigen-analysis of section II-D predicts: after 6000 iterations it is 10^{-10} for OLS and 10^{-12} for Ridge, whose better conditioning gives the steeper slope. The runs with analytical and automatically differentiated gradients are indistinguishable, as expected from their agreement to 5×10^{-16} (section III-C).

Figure 13 tests the stability bound eq. (19). After 2000 steps the cost gap decreases steadily as γ approaches $2/\lambda_{\max}(H)$ and jumps to divergence immediately beyond it, for both OLS and Ridge. The best fixed learning rate lies just below the bound: the optimal constant step for a quadratic is $2/(\lambda_{\max} + \lambda_{\min})$, which is within 0.2% of $2/\lambda_{\max}$ when $\kappa \approx 600$.

Table I compares the five update rules by the number of iterations needed to reach $\|\theta - \hat{\theta}\|_2 / \|\hat{\theta}\|_2 < 10^{-3}$, with the learning rate tuned for each optimizer and problem on a logarithmic grid of 25 values between 10^{-5} and 10 (four per decade), and a cap of 10^5 iterations. Figure 14 shows the full dependence on the learning rate for two of the problems. Three observations stand out.

First, momentum and Adam are an order of magnitude faster than the other update rules. At $p = 5$, momentum needs 126 iterations and Adam 337, against 3166 for plain GD, a factor of 25 and 9.4; AdaGrad and RMSprop are no faster than plain GD. For the Ridge problems at $p = 10$ and 15 the two are equally fast within the resolution of the learning-rate grid (1342 vs. 1422 and 1913 vs. 1379 iterations), about 18 times faster than plain GD. Momentum also converges for learning rates beyond the plain-GD bound (fig. 14a); this agrees with the stability bound of the heavy-ball iteration for a quadratic, $\gamma < 2(1 + \beta)/\lambda_{\max} = 0.58$ at $p = 5$.

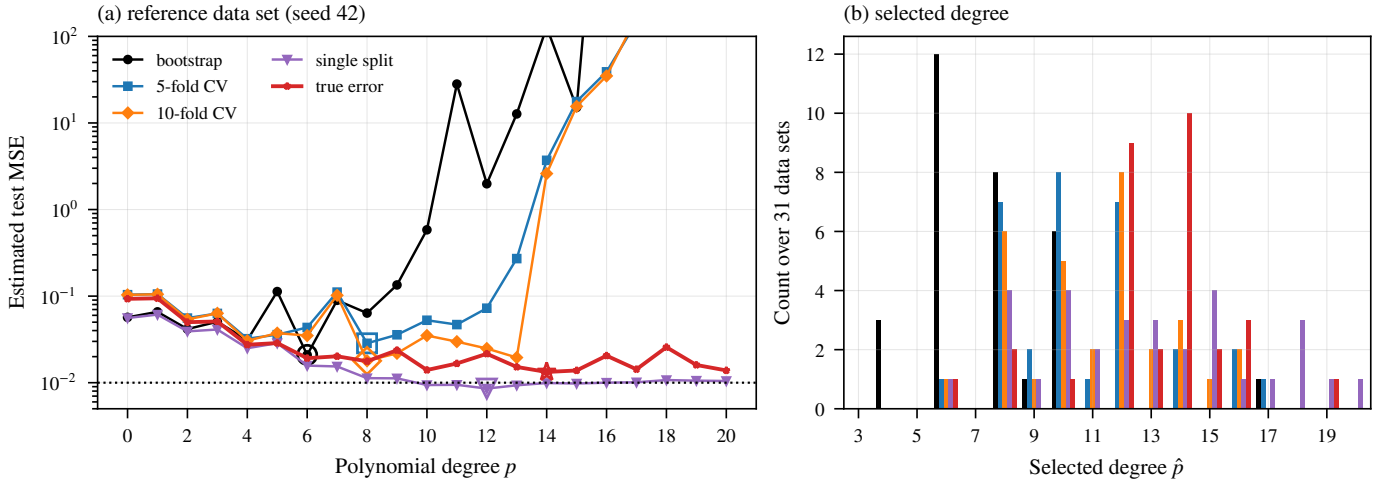


Fig. 9. Estimated OLS test error vs. degree. (a) Reference data set: bootstrap ($B = 100$), 5- and 10-fold CV and a single 80/20 split, all computed from the same 80 training points, and the true error of the fit on those 80 points (from the known f). Open markers: minima. (b) Degree selected by each estimator over 31 independent data sets.

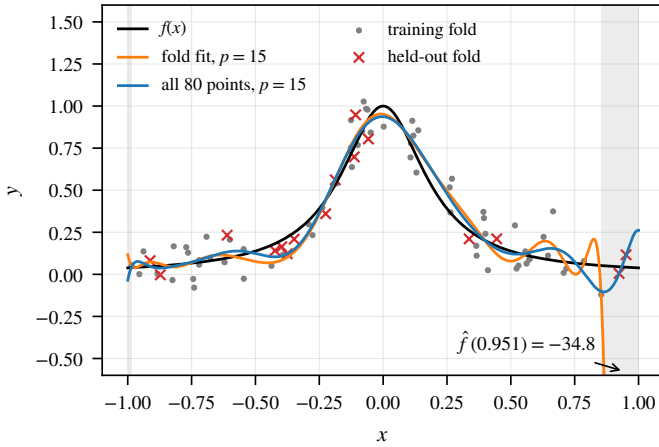


Fig. 10. The 5-fold CV split that dominates the CV estimate at $p = 15$ (reference data). Orange: fit on the four training folds (gray points), which must extrapolate beyond their range (shaded) to the held-out point at $x = 0.951$. Blue: fit on all 80 training points.

Second, Adam is by far the least sensitive to the learning rate: it reaches the tolerance for all 25 learning rates at $p = 5$ and for 23 of 25 at $p = 10$ and 15, and for $\gamma \gtrsim 0.03$ its iteration count is essentially independent of γ . Plain GD converges for only 3–7 of the 25 values, and momentum for 8–13; both need γ within a factor of a few of the bound eq. (19). AdaGrad and Adam are insensitive to large γ because they normalize the step by the gradient magnitude. RMSprop, which does the same with a short memory, only converges for $\gamma \leq 6 \times 10^{-4}$: at larger γ its normalized steps keep it oscillating around the minimum.

Third, the conditioning decides everything. For OLS at $p = 10$ and 15, with $\kappa(\mathbf{H}) = 2.8 \times 10^6$ and 2.6×10^{10} , no optimizer reaches the tolerance within 10^5 iterations; the best relative error at $p = 10$ is 0.28 (Adam), and at $p = 15$ none gets below

0.99, barely closer to $\hat{\theta}$ than the starting point $\theta = \mathbf{0}$ (error 1). With $\gamma = 1/L$, plain GD would need about $\kappa \ln 10^3 \approx 2 \times 10^7$ iterations at $p = 10$. A penalty of only $\lambda = 10^{-3}$, which barely changes the fit, reduces κ to $\sim 10^4$ (section II-D), and all optimizers converge again. Where they converge, the iteration counts of plain GD scale with κ as predicted: $3166 \approx 5\kappa$ at $p = 5$ and $26244 \approx 4\kappa$ for Ridge at $p = 10$. The OLS parameters at high degree are, however, ill-determined anyway (fig. 3): a relative parameter error of order one does not preclude a good prediction.

F. Lasso

We solve Lasso with our gradient-descent code, using the subgradient eq. (27) with each of the update rules of section IV-E, and compare with scikit-learn’s coordinate-descent Lasso, whose solution defines the optimal cost C^* . Figure 15 shows the convergence at two configurations of the reference data: the one selected by 5-fold CV ($p = 6$, $\lambda = 10^{-2.5}$), and a high-degree one ($p = 15$, $\lambda = 10^{-4}$).

At $p = 6$, where the smooth part of the cost has $\kappa = 2.3 \times 10^3$, all update rules approach C^* , but none converges in the usual sense. Each levels off at a gap set by its step size, as expected for the subgradient method with a fixed step (section II-D): momentum at 8×10^{-6} after a few thousand iterations, RMSprop at 4×10^{-4} , and Adam, whose normalized step does not shrink near the minimum, oscillates between 10^{-4} and 10^{-2} and is at 2.4×10^{-3} after 5×10^4 iterations, worse than after 5000 (8.7×10^{-4}). Plain GD and AdaGrad, whose effective steps are smallest near the end, reach 10^{-6} . The coefficients from our default setting (Adam, 5000 iterations) differ from scikit-learn’s by at most 4×10^{-3} , but scikit-learn sets the coefficient of x^5 exactly to zero, while ours is -3.8×10^{-3} : our solver produced no exact zeros in any of the fits we examined.

The analytical subgradient uses $\text{sign}(0) = 0$; the gradient from JAX of $|\theta_j|$ at exactly $\theta_j = 0$ is 1 (section III-C). Both

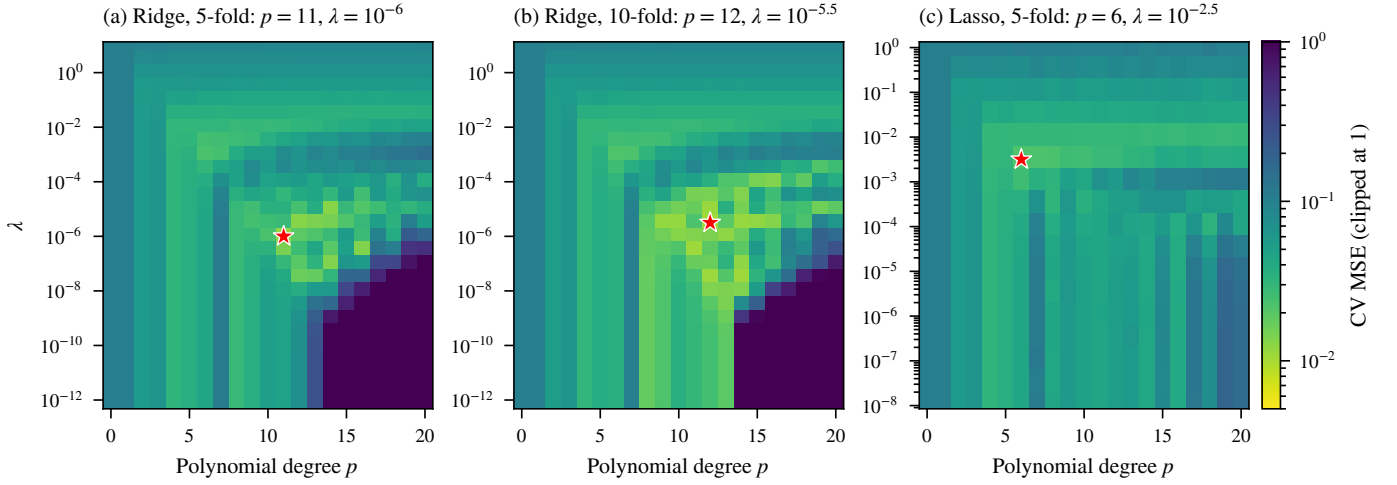


Fig. 11. CV test MSE over polynomial degree and penalty on the reference training set: Ridge with (a) 5-fold and (b) 10-fold CV, and (c) Lasso (our subgradient solver, Adam, $\gamma = 0.05$, 5000 iterations) with 5-fold CV. Stars: minima. Colors are clipped at 1.

TABLE I

ITERATIONS TO REACH THE CLOSED-FORM OLS OR RIDGE ($\lambda = 10^{-3}$) SOLUTION TO A RELATIVE PARAMETER ERROR OF 10^{-3} , FROM $\theta = \mathbf{0}$, WITH THE LEARNING RATE TUNED PER OPTIMIZER AND PROBLEM (REFERENCE TRAINING SET, STANDARDIZED FEATURES, DEGREE p). $\kappa(\mathbf{H})$ IS THE CONDITION NUMBER OF THE HESSIAN. IN PARENTHESES: THE NUMBER OF THE 25 LEARNING RATES FOR WHICH THE TOLERANCE WAS REACHED WITHIN 10^5 ITERATIONS. “—”: NOT REACHED FOR ANY LEARNING RATE; IN BRACKETS THE SMALLEST RELATIVE ERROR REACHED. BOLD: FASTEST.

p	model	$\kappa(\mathbf{H})$	plain GD	momentum	AdaGrad	RMSprop	Adam
5	OLS	6.0×10^2	3194 (6)	126 (12)	3383 (14)	3188 (8)	339 (25)
5	Ridge	5.1×10^2	2639 (7)	114 (13)	2807 (14)	2796 (8)	295 (25)
10	OLS	2.8×10^6	—[0.92]	—[0.41]	—[0.92]	—[0.90]	—[0.28]
10	Ridge	6.4×10^3	26302 (3)	1425 (8)	29305 (12)	20726 (7)	1344 (23)
15	OLS	2.6×10^{10}	—[1.00]	—[1.00]	—[1.00]	—[1.00]	—[1.00]
15	Ridge	9.8×10^3	25375 (3)	1382 (8)	43280 (12)	29115 (7)	1917 (23)

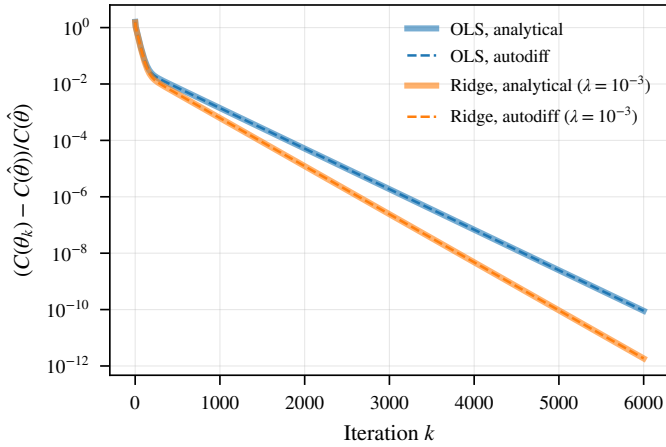


Fig. 12. Plain gradient descent with $\gamma = 1/L$ at $p = 5$ (reference training set): relative gap between the cost and the cost of the closed-form solution vs. iteration, for OLS and Ridge ($\lambda = 10^{-3}$), with analytical (thick) and JAX (dashed) gradients.

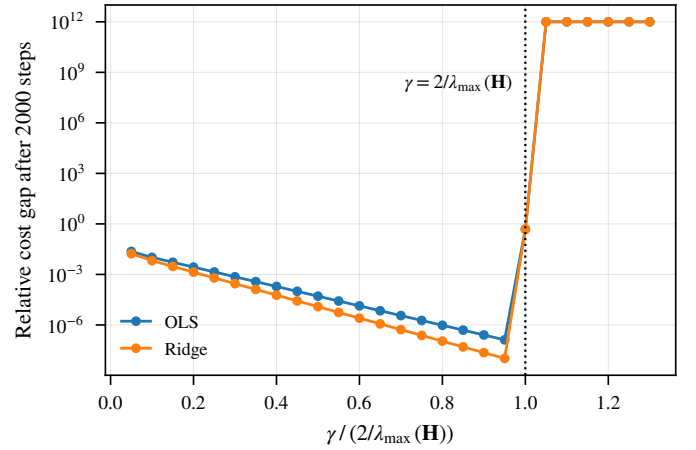


Fig. 13. Relative cost gap after 2000 plain-GD steps vs. the learning rate in units of the bound $2/\lambda_{\max}(\mathbf{H})$ ($p = 5$). Diverged runs are shown at 10^{12} .

are valid subgradients, since $\partial|\theta_j| = [-1, 1]$ at zero, but the choice rarely matters in practice: the iterates are almost never exactly zero, which is the reason for the oscillation in the first place.

At $p = 15$, with $\kappa = 2.6 \times 10^{10}$, the picture is much worse (fig. 15b). After 5×10^4 iterations Adam is at a gap of 1.1×10^{-2} , momentum at 2.5×10^{-2} , and the others above 0.1; scikit-learn’s coordinate descent itself needs 4×10^5 passes. Figure 16 compares the coefficients: scikit-learn sets five of the

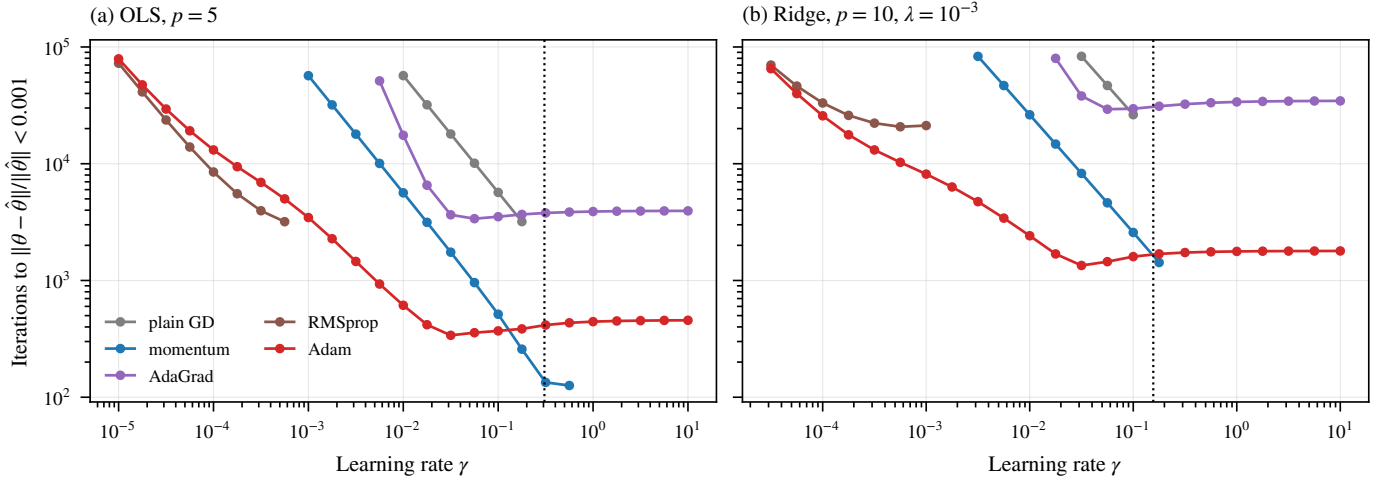


Fig. 14. Iterations to reach the closed-form solution to a relative parameter error of 10^{-3} vs. the learning rate, for each optimizer. Missing points: tolerance not reached within 10^5 iterations, or divergence. Dotted: the plain-GD stability bound $2/\lambda_{\max}(\mathbf{H})$.

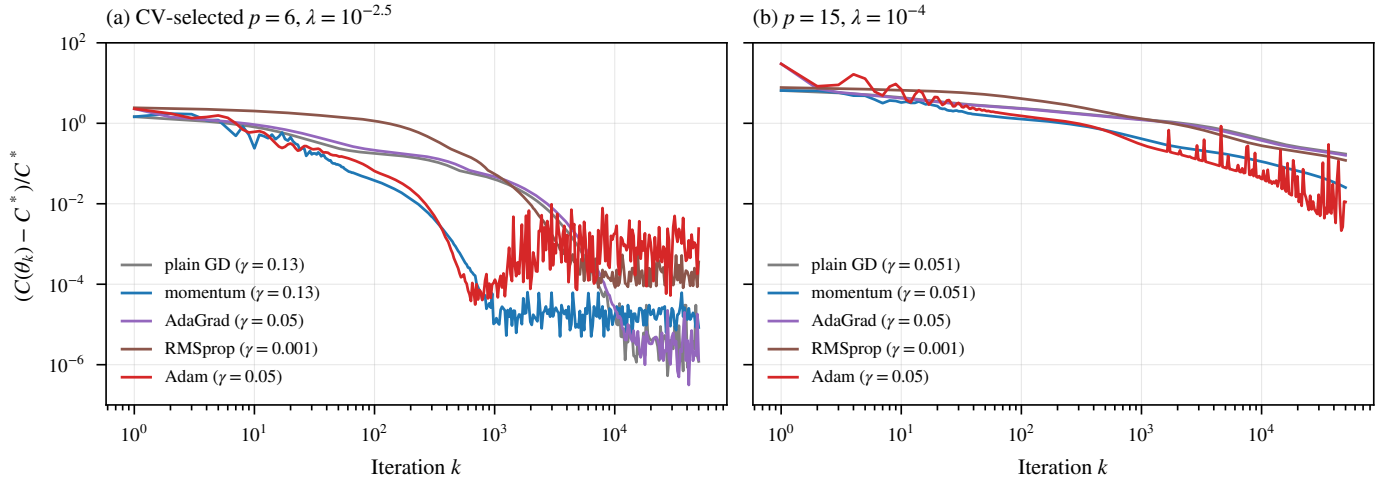


Fig. 15. Relative gap between the Lasso cost of subgradient descent and the cost C^* of scikit-learn's solution vs. iteration, from $\theta = 0$, for each update rule (reference training set). (a) The configuration selected by 5-fold CV, $p = 6$, $\lambda = 10^{-2.5}$. (b) $p = 15$, $\lambda = 10^{-4}$. Plain GD and momentum use $\gamma = 1/L$.

15 to zero, while ours after 5000 iterations differ by up to 4.8 and are as large as 1.6 where scikit-learn's are zero; even after 5×10^4 iterations they differ by up to 0.5. Our Lasso fits at high degree are therefore early-stopped gradient descent rather than Lasso solutions. This explains the Lasso CV surface of fig. 11c: at high degree the fits are poor regardless of λ , so CV prefers a low degree, where the solver converges.

G. Stochastic gradient descent

Figure 17 and table II compare mini-batch SGD with full-batch GD for OLS at $p = 5$, at equal computational cost measured in floating-point operations of the gradient evaluations: one epoch of SGD costs as much as one full-batch step. Plain SGD uses $\gamma_0 = 1/L$ and Adam $\gamma_0 = 0.03$ (the best fixed rate of table I), both with the decay $\gamma_t = \gamma_0/(1+10^{-3}t)$.

With plain updates, small mini-batches make faster progress per FLOP early on, because they make $80/M$ updates per

epoch instead of one: after 200 epochs the relative cost gap is 2.7×10^{-3} with $M = 4$ and 2.3×10^{-2} with the full batch. Later, the gradient noise sets a floor: all batch sizes end between 9×10^{-5} and 6×10^{-4} after 2000 epochs, fluctuating from epoch to epoch, while full-batch GD reaches 5.1×10^{-5} monotonically. With Adam the picture is reversed: full-batch Adam reaches the closed-form cost to machine precision within about 10^6 FLOPs, whereas mini-batch Adam stalls at 10^{-5} – 10^{-3} . Full-batch Adam also shows the behavior noted in section II-D: after converging, its fixed step size produces a brief excursion (the spike in fig. 17b) before it settles again.

Table II also shows the price of SGD in practice. With equal FLOPs, SGD takes three to four times longer in wall-clock time, because each of its five times more numerous updates carries a fixed Python overhead that is large compared with the arithmetic for 16 rows. For a problem with 80 training points

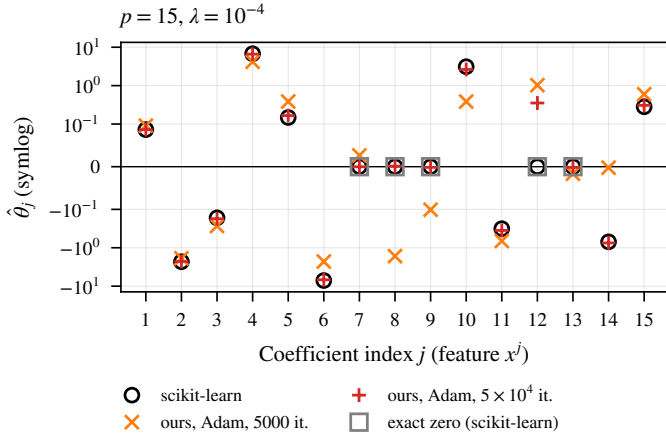


Fig. 16. Lasso coefficients of the standardized features at $p = 15$, $\lambda = 10^{-4}$ (reference training set): scikit-learn (circles; squares mark exact zeros) and our subgradient descent with Adam after 5000 and 5×10^4 iterations.

TABLE II
FULL-BATCH GD VS. MINI-BATCH SGD ($M = 16$) AFTER 2000 EPOCHS
AT $p = 5$: NUMBER OF UPDATES, FLOPS OF THE GRADIENT
EVALUATIONS, FINAL RELATIVE COST GAP, AND MEDIAN WALL-CLOCK
TIME OF THREE RUNS.

model	optimizer	batch	updates	FLOPs	rel. gap	ms
OLS	plain	full	2000	3.2×10^6	5.1×10^{-5}	14
OLS	plain	16	10000	3.2×10^6	8.5×10^{-5}	44
OLS	Adam	full	2000	3.2×10^6	$< 10^{-15}$	21
OLS	Adam	16	10000	3.2×10^6	2.0×10^{-5}	77
Ridge	plain	full	2000	3.2×10^6	1.2×10^{-5}	14
Ridge	plain	16	10000	3.2×10^6	6.9×10^{-5}	45
Ridge	Adam	full	2000	3.2×10^6	$< 10^{-15}$	20
Ridge	Adam	16	10000	3.2×10^6	2.6×10^{-5}	77

and 6 parameters, SGD thus has no advantage: its benefit, cheap updates when n is large and the gradient is expensive, does not apply, and its noise prevents it from reaching the accuracy of full-batch methods without a carefully tuned decay of the learning rate.

H. Final comparison of OLS, Ridge and Lasso

The comparison on the reference data rests on one set of folds and one test set of 20 points. To compare OLS, Ridge and Lasso reliably, we repeated the whole selection on 31 independent data sets: 5-fold CV on the 80 training points over the grids of fig. 11, a refit of the selected configuration on all 80 points, and evaluation on the 20 held-out points and, using the known f , of the true error. Figure 18 and table III summarize the results.

Judged by the CV error itself, Ridge beats OLS in 25 of the 31 data sets, by a median of 5%. This is the number a naive comparison would report, and it is biased: the CV minimum of Ridge is taken over a two-dimensional grid, that of OLS over a one-dimensional one, and the minimum of noisy estimates over more candidates is lower even when no candidate is better. On held-out data the advantage disappears: Ridge beats OLS on the test set in 15 of 31 data sets, with a median relative difference of 0.0%, and in the true error in

TABLE III
MODEL SELECTION OVER 31 DATA SETS: MEDIAN CV, TEST AND TRUE MSE OF THE CV-SELECTED MODEL, MEDIAN SELECTED DEGREE, AND THE NUMBER OF DATA SETS IN WHICH THE METHOD BEATS OLS.

	CV MSE	test MSE (median)	true MSE	$\hat{p}$ (median)	better than OLS CV/test/true
OLS	0.0139	0.0134	0.0146	10	—
Ridge	0.0129	0.0136	0.0137	12	25/15/19
Lasso	0.0150	0.0134	0.0156	8	11/12/10

19 of 31, with a median of -0.2% . The penalty buys stability rather than accuracy: it lets CV select a higher degree (median 12 instead of 10), with a smaller bias, without an increase in variance. The means are dominated by a few data sets in which the selected model extrapolates badly: Ridge with $p = 15$ and $\lambda = 10^{-10}$ reaches a true MSE of 18 on one data set, where all of the error comes from x beyond the largest training input.

Lasso, fitted with our subgradient solver, is worse than OLS by every measure: it beats OLS in only 11 (CV), 12 (test) and 10 (true error) of the 31 data sets, with a median relative increase of its true error of 13%. CV selects a low degree for it (median 8), for the reason found in section IV-F: at high degree the solver does not reach the Lasso solution within 5000 iterations, so these fits are poor and CV avoids them. None of the selected Lasso models has an exact zero coefficient. This comparison therefore measures our optimizer as much as the Lasso model; a solver that converges at high degree could change the ranking.

Which term of the bias–variance decomposition does each method attack? OLS controls the variance only through the degree. Ridge reduces the variance directly by shrinking the small-singular-value directions (fig. 5), at the cost of bias along them, which allows a more flexible, less biased model for the same variance. Lasso aims at the same term by setting coefficients exactly to zero, but our fits never reach that regime; what limits them in practice is the incomplete optimization, which acts like early stopping and adds bias. Our numerical evidence supports the reading for Ridge qualitatively (higher selected degrees, flat error curves in fig. 7), but it also shows its limits for this problem. The selected Ridge penalties are tiny, $\lambda \in [10^{-12}, 3 \times 10^{-5}]$ (5 of 31 at the lower end of the grid, i.e. effectively OLS), because with $n = 100$ and $\sigma = 0.1$ the remaining error of a well-chosen OLS fit is dominated not by variance in the interior of the interval but by the extrapolation near its ends, which a global penalty barely touches. For Runge’s function with a random design, no method is thus significantly better than OLS on the held-out data we would have in practice, and Ridge is better by less than 1% in the true error.

V. CONCLUSION

We fitted polynomials of degree up to 20 to noisy samples of Runge’s function with OLS, Ridge and Lasso, selected models with the bootstrap and k -fold cross-validation, and solved the regression problems with our own gradient-descent code. The central finding is that the resampling methods, not the regression methods, limit model selection here. With

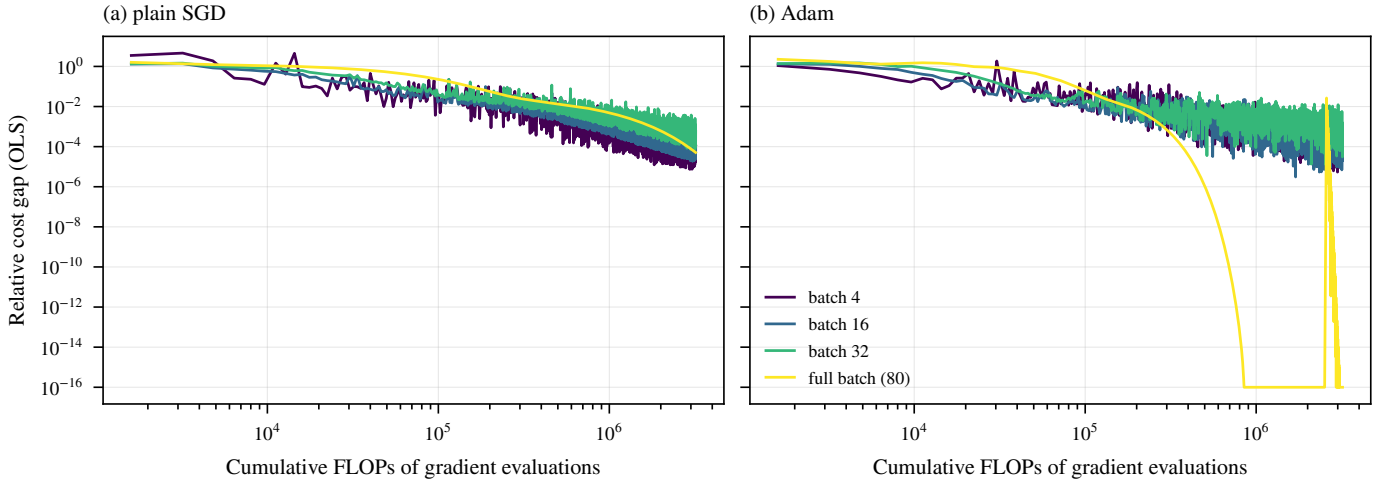


Fig. 17. Relative cost gap (OLS, $p = 5$) vs. cumulative FLOPs of the gradient evaluations for mini-batch sizes $M \in \{4, 16, 32\}$ and the full batch ($M = 80$), over 2000 epochs, for (a) plain SGD with $\gamma_0 = 1/L$ and (b) Adam with $\gamma_0 = 0.03$; SGD with the decay $\gamma_t = \gamma_0/(1 + 10^{-3}t)$. Values below 10^{-16} are shown at 10^{-16} .

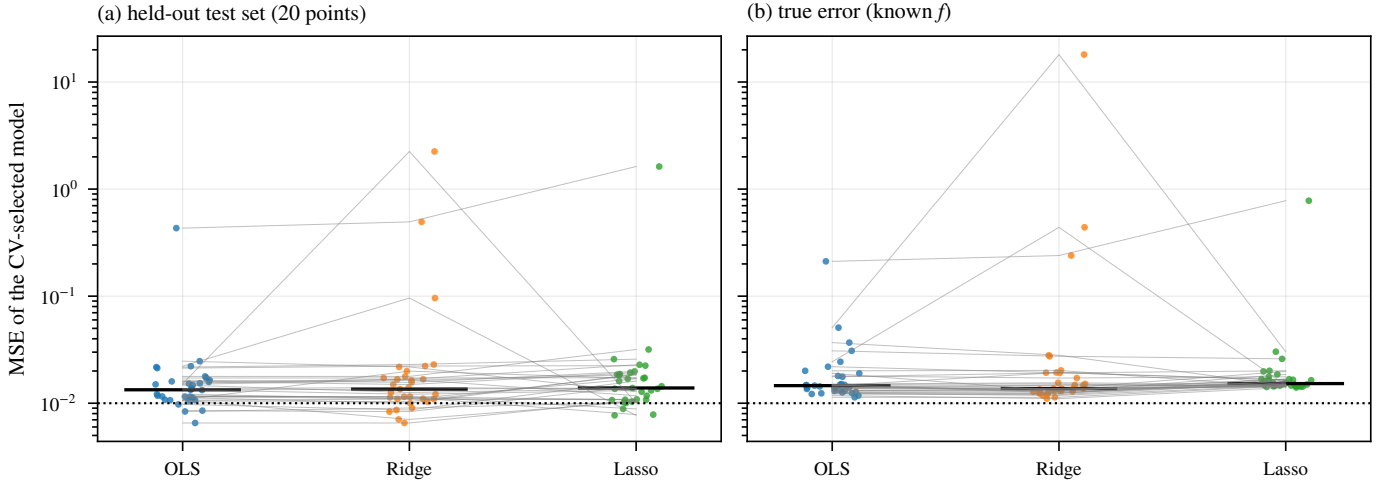


Fig. 18. Error of the model selected by 5-fold CV for each method on 31 independent data sets ($n = 100$, $\sigma = 0.1$): (a) on the 20 held-out test points, (b) true error from the known f . Gray lines connect the same data set; black bars are medians; dotted: σ^2 .

$n = 100$ random inputs, the true test error of OLS is flat between $p \approx 10$ and 20, but the bootstrap and CV estimates grow by orders of magnitude at high degree, because every resampled fit must extrapolate to held-out points outside its training range, and a high-degree polynomial diverges there, a statistical echo of Runge’s phenomenon. The estimators therefore select too low a degree (median 8 for the bootstrap, 10–12 for CV, against 14 for the true optimum), the more so the fewer distinct points each fit uses. Ridge suppresses the directions of small singular values that cause these failures, which lets CV choose higher degrees with the same variance, but the selected penalties are tiny and the gain is marginal: the CV error drops by a median of 5%, the held-out error not at all, and the true error by 0.2%. Lasso, fitted by subgradient descent, did worse than OLS, because our solver does not reach the Lasso solution at high degree. On the optimization side, the condition number of the Hessian, which grows

exponentially with the degree, explains all our results: plain GD needs about κ iterations, momentum and Adam reach the closed-form solution 25 and 9 times faster at $p = 5$, no first-order method reaches the OLS solution at $p \geq 10$, a Ridge penalty of 10^{-3} restores convergence, and subgradient descent for Lasso levels off at a distance from the optimum set by its step size and never produced an exact zero.

Our study has clear limitations. The true error, which we used to judge the estimators, requires the known f and is unavailable in practice; the same holds for the bias against f . We studied a single noise level and sample size for the model selection, a uniform random design, and one set of folds per data set, so the size of the extrapolation effect is specific to these choices; on an equispaced or Chebyshev grid, which contain the endpoints of the interval, it would be much smaller. The bootstrap estimates of bias and variance are means over replicates and are dominated by the few

replicates that extrapolate badly; a robust estimate (median) would separate the two effects. The optimizer comparison used a learning-rate grid with four values per decade and fixed momentum and Adam parameters, which is too coarse to rank momentum and Adam where they differ by less than a factor of two, and it measures iterations, not time. Our Lasso results are limited by the solver: with a fixed number of 5000 Adam steps, the high-degree Lasso fits in the cross-validation are far from converged, so the Lasso model selection and comparison measure our optimizer as much as the Lasso model itself. Finally, all comparisons on the held-out data rest on 20 test points, whose MSE has a standard error of about 30% of σ^2 .

Two extensions follow directly. First, replacing the monomials x^j by orthogonal polynomials (Legendre or Chebyshev) on $[-1, 1]$ spans the same model space but keeps $\kappa(\mathbf{X})$ small; gradient descent, including the subgradient method for Lasso, would then converge at every degree, and the coefficients would become interpretable. Second, we would make the resampling estimates less sensitive to extrapolation, for instance by assigning the smallest and largest x to the training part of every fold, or by aggregating fold errors with the median, and test whether the selected degree then approaches the true optimum. Both are needed before the conclusions of this study can be transferred to more advanced techniques such as neural networks, whose optimization is equally sensitive to conditioning.

REFERENCES

- [1] K. Weierstrass, “Über die analytische Darstellbarkeit sogenannter willkürlicher Functionen einer reellen Veränderlichen,” *Sitzungsberichte der Akademie zu Berlin*, pp. 633–63, 789–805, 1885.
- [2] M. Hjorth-Jensen, *Machine Learning and the Physical Sciences: From Discriminative Methods to Generative Methods and Reinforcement Learning*. Springer, Oct. 9, 2026. [Online]. Available: <https://github.com/EducationalMaterialUiO/MLearningUiO/blob/main/doc/BookML/Lecturebook.pdf>
- [3] T. Hastie, R. Tibshirani, and J. H. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (Springer Series in Statistics), Second edition. New York, NY: Springer, 2017, ISBN: 978-0-387-84857-0.
- [4] C. Runge, “Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten,” *Zeitschrift für Mathematik und Physik*, **46**, pp. 224–243, 1901.
- [5] S. De Marchi, F. Marchetti, E. Perracchione, and D. Poggiali, “Polynomial interpolation via mapped bases without resampling,” *Journal of Computational and Applied Mathematics*, **364**, p. 112 347, Jan. 2020, ISSN: 03770427. DOI: 10.1016/j.cam.2019.112347 Accessed: Sep. 17, 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0377042719303449>
- [6] A. E. Hoerl and R. W. Kennard, “Ridge Regression: Biased Estimation for Nonorthogonal Problems,” *Technometrics*, **12**, no. 1, pp. 55–67, Feb. 1, 1970, ISSN: 0040-1706. DOI: 10.1080/00401706.1970.10488634
- Accessed: Sep. 22, 2026. [Online]. Available: <https://doi.org/10.1080/00401706.1970.10488634>
- [7] R. Tibshirani, “Regression Shrinkage and Selection Via the Lasso,” *Journal of the Royal Statistical Society: Series B (Methodological)*, **58**, no. 1, pp. 267–288, Jan. 1, 1996, ISSN: 0035-9246. DOI: 10.1111/j.2517-6161.1996.tb02080.x Accessed: Sep. 22, 2026. [Online]. Available: <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>
- [8] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization.” arXiv: 1412.6980 [cs.LG], Accessed: Sep. 21, 2026. [Online]. Available: <http://arxiv.org/abs/1412.6980> pre-published.
- [9] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, “Automatic Differentiation in Machine Learning: A Survey,” *Journal of Machine Learning Research*, **18**, pp. 1–43, 2018. [Online]. Available: <https://jmlr.org/papers/v18/17-468.html>
- [10] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, **12**, no. 85, pp. 2825–2830, 2011. [Online]. Available: <http://jmlr.org/papers/v12/pedregosa11a.html>
- [11] J. Bradbury et al., *JAX: Composable transformations of Python+NumPy programs*, version 0.3.13, 2018. [Online]. Available: <http://github.com/jax-ml/jax>
- [12] S. Riseth, *FYS-STK4155 Project 1*, version v0.1.0, GitHub, Oct. 2026. [Online]. Available: <https://github.com/sigurdriseth/FYS-STK4155-P1>
- [13] P. Mehta et al., “A high-bias, low-variance introduction to Machine Learning for physicists,” *Physics Reports*, **810**, pp. 1–124, May 2019, ISSN: 03701573. DOI: 10.1016/j.physrep.2019.03.001 Accessed: Aug. 26, 2026. [Online]. Available: <http://arxiv.org/abs/1803.08823>
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. Accessed: Aug. 12, 2026. [Online]. Available: <https://www.deeplearningbook.org/>
- [15] B. Efron, “Bootstrap Methods: Another Look at the Jackknife,” *The Annals of Statistics*, **7**, no. 1, pp. 1–26, Jan. 1979, ISSN: 0090-5364, 2168-8966. DOI: 10.1214/aos/1176344552 Accessed: Sep. 23, 2026. [Online]. Available: <https://projecteuclid.org/journals/annals-of-statistics/volume-7/issue-1/Bootstrap-Methods-Another-Look-at-the-Jackknife/10.1214/aos/1176344552.full>
- [16] A. Géron, *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems* (Data Science / Machine Learning), Third edition. O’Reilly, 2023, 834 pp., ISBN: 978-1-0981-2597-4.
- [17] J. Duchi, E. Hazan, and Y. Singer, “Adaptive sub-gradient methods for online learning and stochastic optimization,” *Journal of Machine Learning Research*, **12**, no. 61, pp. 2121–2159, 2011. [Online]. Available: <http://jmlr.org/papers/v12/duchi11a.html>

APPENDIX

A. Use of AI/LLM tools

Large language models were used throughout this project, following the course guidelines¹, which also define the contribution levels below. All code was tested, all derivations were checked, and every claim in the report was read, understood and verified by the author.

1) Tools used:

- Anthropic Claude via <https://claude.ai>, Claude Code, and the Claude desktop app (cowork), September–October 2026 (Claude Sonnet 5.0 and Opus 5.5, 5, and 4.8 models).
- OpenAI ChatGPT via <https://chatgpt.com>, September 2026.
- GitHub Copilot via VS Code, unknown model, September 2026.

Claude was used most often, both for code development and for writing the report. ChatGPT was sometimes used to discuss solutions and writing. GitHub Copilot was used only rarely through the VS Code “fix” interface to address minor warnings.

2) *Text writing and editing*: Table IV gives the contribution level for each part of the report.

TABLE IV
LLM CONTRIBUTION TO TEXT WRITING AND EDITING (LEVELS 0–3).

Section	Level	Notes
Abstract	3	Drafted by Claude from the finished results; revised by the author.
Introduction	2	Authors text; last three paragraphs revised by Claude.
Methods	2	Authors text; corrected by Claude.
Implementation	2	Authors text; revised by Claude.
Results and discussion	2	Authors text; revised and improved by Claude.
Conclusion	2	Drafted by the author; revised and improved by Claude.
Appendices	3	Drafted by Claude; revised by the author.

For the abstract, with a contribution level of 3, we gave Claude access to the project repository containing both the source code and the $\text{\LaTeX}$ source of the report. The LLM was instructed to review the implementation, results, and written report before drafting a proposed abstract. We also provided relevant course literature on writing abstracts and scientific reports, which the LLM used as guidance when preparing the draft. The abstract was subsequently reviewed and revised by the author.

The appendices are also assigned level 3 because the LLM was used to help maintain the documentation of AI/LLM use throughout the project. When using an LLM, we instructed it to record its contribution in this appendix, including the relevant section or code file, contribution level, and a brief description of how the LLM was used. This allowed the appendix to be updated as LLM-assisted work was performed rather than reconstructed retrospectively. The final appendix

was reviewed and revised by the author to ensure that the reported contribution levels and descriptions accurately reflect the work performed.

3) *Code generation and assistance*: Table V gives the level for each source file (levels 0–4). Functions with level 2 or above carry an LLM-assisted tag in their docstring. Claude Code was also used to write docstrings and unit tests across the code base.

4) *Declared and undeclared use*: An example of a use that must be declared is the benchmark function `iterations_to_tolerance` in `benchmark.py`: Claude wrote the function, which appears in our submission, so it is declared here and in its docstring. An example of a use that need not be declared is using an LLM as a tutor, for instance to explain the difference between a gradient and a subgradient, and then writing about it in our own words.

B. Additional results

Figures 19 and 20 repeat the bootstrap decomposition of fig. 8 for other sample sizes and noise levels (seed 42, $B = 100$, 80/20 split). More data lowers the variance at every degree, while the bias, a property of the model class, hardly changes; the minimum therefore moves from $p = 4$ at $n = 50$ to $p = 12$ at $n = 400$, and for $n = 400$ the variance stays below σ^2 up to $p \approx 16$. More noise scales up the variance and the floor σ^2 but leaves the bias against f essentially unchanged, so noisier data favor simpler models. For $n \leq 100$ the curves beyond the minimum are irregular, for the extrapolation reason discussed in section IV-D.

¹https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/LLM_Usage_Declaration_Guidelines.md

TABLE V
LLM CONTRIBUTION TO THE CODE (LEVELS 0–4).

File	Level	Description
design_matrix.py, runge.py, metrics.py	0	
ordinary_least_squares.py	1	Claude suggested the singular-value cutoff of the pseudoinverse.
ridge.py, cost.py, shrinkage.py	0	
base.py	2	scikit-learn estimator interface.
autodiff.py	2	JIT compilation.
optimizers.py	3	Provided the framework of the <code>Optimizer</code> class and shape validation.
schedules.py	4	Learning-rate schedules; reviewed and tested by us.
gradient_descent.py	2	Learning-rate scheduling.
degree_sweep.py	3	Extracted and refactored from our notebooks and generalized.
lasso.py	0	
bootstrap.py	2	Bias against the known f added by Claude.
cross_validation.py	2	Per-fold output added by Claude.
benchmark.py	4	Iterations-to-tolerance benchmark, written by Claude.
scripts/	3	Experiments, figures and configuration, written in part by Claude, inspired by the notebooks.
notebooks/	2	Assistance with plotting, providing feedback, and debugging.
tests/	3	Tests for the new code written by Claude; others with Claude Code.

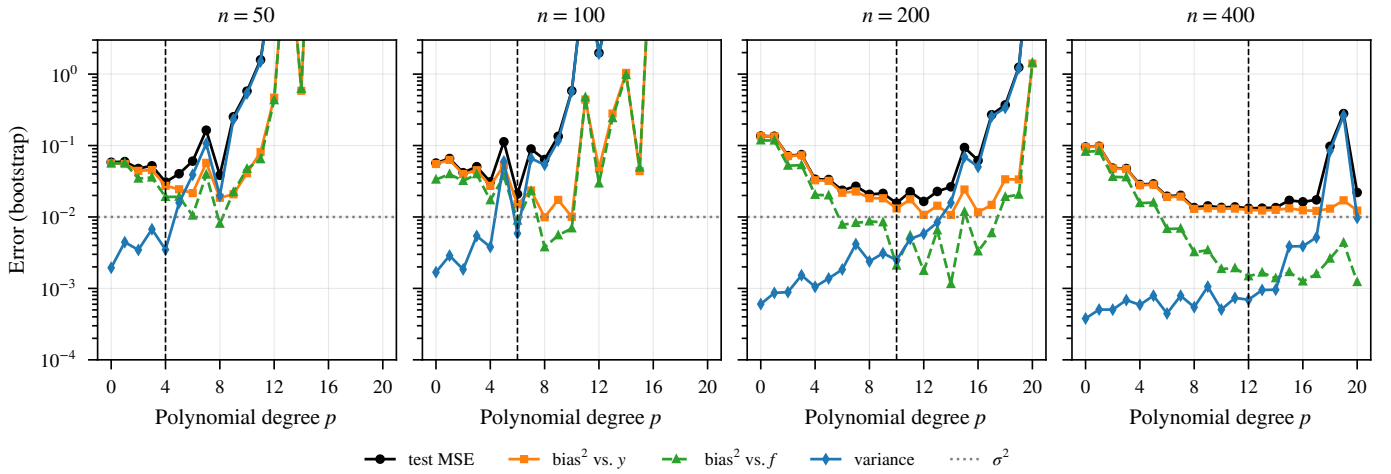


Fig. 19. Bootstrap bias–variance decomposition of the OLS test error for $n \in \{50, 100, 200, 400\}$ at $\sigma = 0.1$. Dashed: minimum of the test MSE. Values above 3 are cut off.

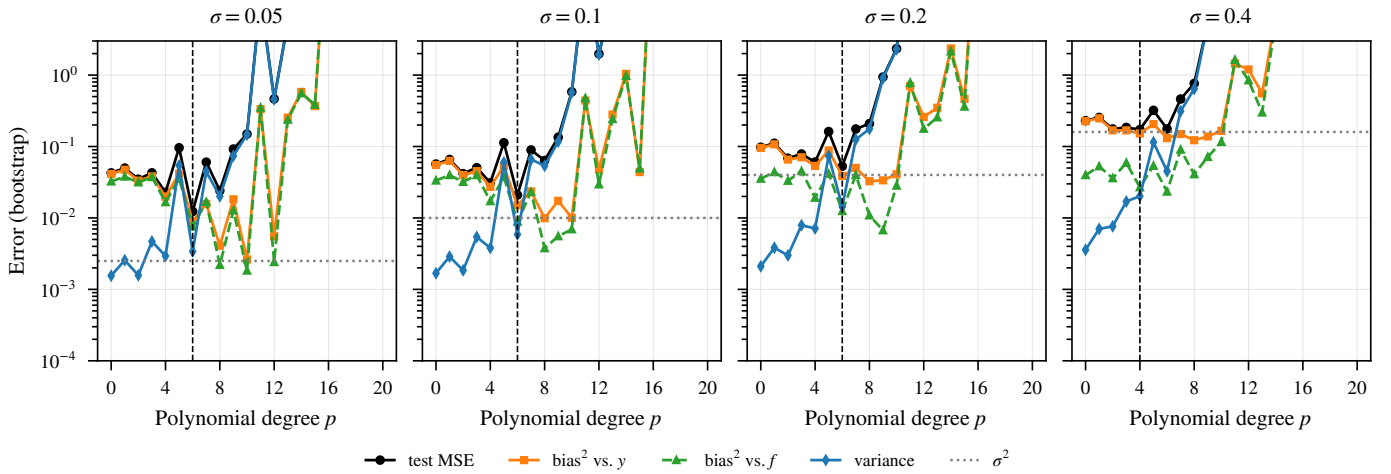


Fig. 20. Bootstrap bias–variance decomposition of the OLS test error for $\sigma \in \{0.05, 0.1, 0.2, 0.4\}$ at $n = 100$.